

IVISS — Deployment & Release Document

Document Title: IVISS Deployment & Release Runbook

Version: 1.0

Date: May 2026

Classification: Internal / Client

Authors: IVISS Development Team

Table of Contents

The sections below are color-coded in **blue** for navigation.

1. [Executive Summary](#)
 2. [System Architecture](#)
 3. [Hosting & Infrastructure](#)
 4. [Environments](#)
 5. [CI/CD Pipeline & Release Process](#)
 6. [Versioning & Release Strategy](#)
 7. [Deployment Process](#)
 8. [Configuration & Secrets Management](#)
 9. [Security & Compliance](#)
 10. [Monitoring, Logging & Observability](#)
 11. [Backup & Disaster Recovery](#)
 12. [Rollback & Incident Response](#)
 13. [Maintenance & Operational Procedures](#)
 14. [Troubleshooting](#)
 15. [Appendices](#)
-

1. Executive Summary

IVISS (Integrated Vehicle Inspection and Surveillance System) is a multi-tenant platform designed for law enforcement and regulatory agencies to perform roadside vehicle inspections, verify compliance status, manage enforcement actions, and maintain a centralised vehicle registry.

The system is composed of a mobile-first Progressive Web App for field agents, a web back-office for administrators and supervisors, and a Rust-based API backend. It is deployed on AWS Lightsail using a fully automated CI/CD pipeline built on GitHub Actions, with infrastructure managed as code via Terraform and Ansible.

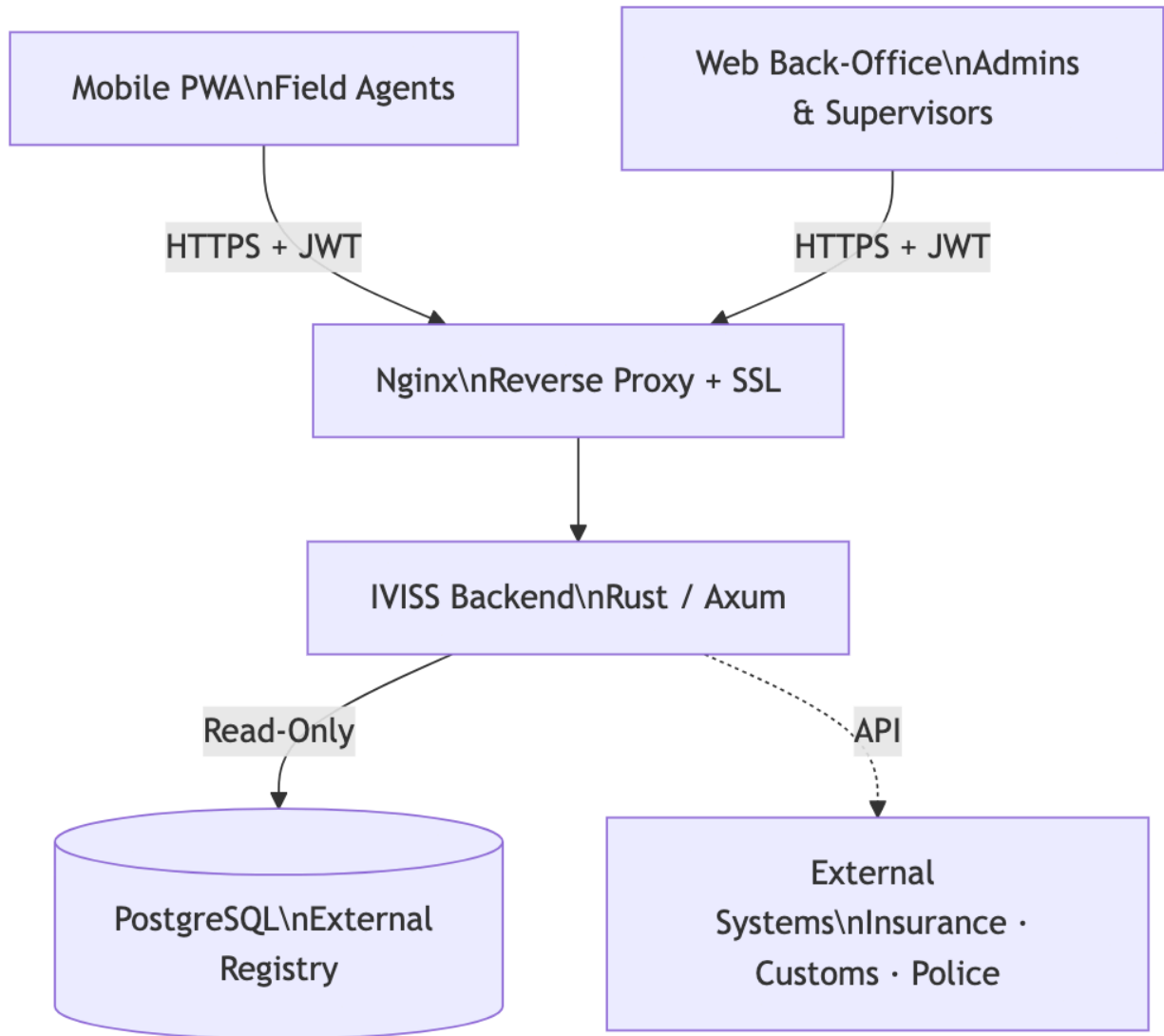
Key technologies:

Layer	Technology
Backend	Rust, Axum, SQLx
Frontend	React, TypeScript, Vite
Database	PostgreSQL 15
Containerisation	Docker, Docker Compose
Infrastructure as Code	Terraform
Configuration Management	Ansible
CI/CD	GitHub Actions
Container Registry	GitHub Container Registry (GHCR)
Cloud Provider	AWS Lightsail

2. System Architecture

2.1 High-Level Overview

The diagram below shows how the different components of IVISS interact. Client applications communicate with the backend through a secure Nginx reverse proxy. The backend handles all business logic and queries both an internal database and an external national registry.



2.2 Component Breakdown

The table below describes the role of each component in the system.

Component	Description
Nginx	Reverse proxy, SSL termination, static file serving for the frontend
IVISS Backend	Core API — handles authentication, vehicle lookups, control records, OTP, RBAC
PostgreSQL (External)	National vehicle registry — the single database used for all vehicle data
GHCR	Private Docker image registry hosted on GitHub

3. Hosting & Infrastructure

This section describes the cloud provider, server specifications, and the tools used to provision and manage the infrastructure automatically.

3.1 Cloud Provider & Server

Resource	Value
Provider	AWS Lightsail
Region	eu-west-1 (Europe — Ireland)
Bundle	small_3_0 — 2 vCPUs, 2 GB RAM, 60 GB SSD
OS	Ubuntu 22.04 LTS
Static IP	Yes — attached via Lightsail static IP resource
Open ports	22 (SSH), 80 (HTTP), 443 (HTTPS)

3.2 Infrastructure as Code

All infrastructure is defined and managed using **Terraform**, ensuring the environment is reproducible and version-controlled. The Terraform state is stored remotely in AWS S3 with DynamoDB locking to prevent concurrent modifications.

IaC Component	Tool	Location
Server provisioning	Terraform	infra/terraform/
Server configuration	Ansible	infra/ansible/
Deployment script	Bash	infra/scripts/deploy.sh
Remote state storage	AWS S3	iviss-terraform-state-<account-id>
State locking	AWS DynamoDB	iviss-terraform-lock (eu-central-1)

3.3 Networking

- All traffic enters through **Nginx** on ports 80/443
 - HTTP is automatically redirected to HTTPS
 - The backend API is not exposed directly — all requests are proxied through Nginx
 - SSL certificates are issued and renewed automatically via **Let's Encrypt / Certbot**
-

4. Environments

IVISS currently operates in two environments. There is no dedicated staging environment — the local development setup serves that purpose.

Environment	Branch	Purpose	URL
Local development	Any feature branch	Developer testing with hot reload	<code>http://localhost:8080</code>
Production	<code>dev</code>	Live system — all merges deploy here	<code>https://<domain></code>

Note: The `dev` branch is the active production branch. There is currently no separate staging environment — the local development environment serves this purpose.

5. CI/CD Pipeline & Release Process

5.1 Tools

Tool	Purpose
GitHub Actions	Pipeline orchestration
Docker Buildx	Multi-platform image builds
GHCR	Docker image storage
Terraform	Infrastructure provisioning
Ansible	Server configuration and app deployment
Semantic Release	Automated versioning and release notes

5.2 Pipeline Stages

Every Pull Request merged into `dev` triggers the following automated sequence:

```
1. CI Checks
  └─ Backend: build, test, coverage, lint, format, security audit
  └─ Frontend: build, lint, type check, unit tests, SonarQube analysis

2. Release
  └─ Semantic Release analyses commits → assigns version → publishes GitHub Release

3. Docker Build & Push (triggered by new release tag)
  └─ Build backend image → push to ghcr.io/skyengpro/iviss/backend:<version>
  └─ Build frontend image → push to ghcr.io/skyengpro/iviss/frontend:<version>

4. Deploy to AWS (triggered after Docker build succeeds)
  └─ Terraform: provision / update infrastructure
  └─ Ansible: configure server, pull new images, restart containers
```

5.3 Branching Strategy

Branch type	Naming convention	Purpose
Feature	feat/123-description	New features
Bug fix	fix/123-description	Bug fixes
Enhancement	enhancement/123-description	Improvements
Infrastructure	dep/description	Deployment/infra changes

All branches are merged into `dev` via Pull Requests. Direct pushes to `dev` require at least one review approval.

5.4 Quality Gates

A deployment only proceeds if all of the following pass:

- Backend unit tests (minimum 50% line coverage enforced)

- Frontend unit tests
 - TypeScript type checking (zero errors)
 - ESLint (maximum 10 warnings)
 - Rust Clippy (zero warnings)
 - Security audit via `cargo-audit`
-

6. Versioning & Release Strategy

6.1 Versioning Scheme

IVISS uses **Semantic Versioning (SemVer)**: `MAJOR.MINOR.PATCH`

Version part	Trigger	Example
PATCH	<code>fix: commits</code>	<code>v0.1.0</code> → <code>v0.1.1</code>
MINOR	<code>feat: commits</code>	<code>v0.1.0</code> → <code>v0.2.0</code>
MAJOR	<code>feat!: or BREAKING CHANGE:</code>	<code>v0.1.0</code> → <code>v1.0.0</code>
No release	<code>chore:, docs:, style:, refactor:</code>	—

6.2 How Versions Are Decided

Version numbers are assigned **automatically** by Semantic Release based on Conventional Commits. No manual decision is required. When a PR is merged, all its commits are analysed and the highest-impact commit type determines the version bump. One release is created per merge regardless of the number of commits.

6.3 Release Artefacts

Each release produces:

- A **Git tag** (e.g. `v0.2.0`) on the `dev` branch
- A **GitHub Release** with auto-generated release notes
- **Docker images** tagged with the version number and pushed to GHCR

7. Deployment Process

7.1 Deployment Method

IVISS uses a **recreate deployment** strategy — existing containers are stopped and replaced with new ones pulling the latest release image. There is no load balancer or blue-green setup at this stage.

Expected deployment duration: 3–8 minutes

7.2 Automated Deployment Steps

The `infra/scripts/deploy.sh` script executes the following:

1. **Terraform init** — initialises the remote backend (S3 + DynamoDB)
2. **Terraform apply** — provisions or updates the Lightsail instance and static IP
3. **SSH key extraction** — saves the generated key for Ansible
4. **Ansible inventory generation** — writes the server IP to the inventory file
5. **SSH readiness check** — waits until port 22 is reachable
6. **Ansible playbook** — configures the server, logs into GHCR, pulls new images, restarts Docker Compose

7.3 Post-Deployment Verification

```
# Check all containers are running
docker compose ps

# Check backend health endpoint
curl https://<domain>/api/v1/health
# Expected: 200 OK

# Check frontend is reachable
curl -I https://<domain>
# Expected: HTTP/2 200
```

7.4 Manual Deployment Trigger

1. Go to GitHub → **Actions** → **Docker**

2. Click **Run workflow** → type `yes` → confirm

8. Configuration & Secrets Management

All sensitive configuration is stored as **GitHub Actions Secrets** — encrypted at rest and injected into the pipeline at runtime. No secrets are stored in the codebase.

8.1 Secret Categories

Category	Secrets
AWS credentials	<code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code>
Domain & SSL	<code>DOMAIN_NAME</code> , <code>CERTBOT_EMAIL</code>
JWT authentication	<code>JWT_SECRET</code> , <code>JWT_PRIVATE_KEY_PEM</code> , <code>JWT_PUBLIC_KEY_PEM</code>
OTP security	<code>ACTIVATION_CODE_PEPPER</code>
Database	<code>POSTGRES_USER</code> , <code>POSTGRES_PASSWORD</code> , <code>POSTGRES_DB</code>
Admin bootstrap	<code>ADMIN_BOOTSTRAP_EMAIL</code> , <code>ADMIN_BOOTSTRAP_PASSWORD</code> , <code>ADMIN_BOOTSTRAP_PHONE</code> , <code>ADMIN_BOOTSTRAP_USERNAME</code>
Container registry	<code>REGISTRY_USERNAME</code> , <code>REGISTRY_TOKEN</code>
SMS provider	<code>SMS_PROVIDER</code> + provider-specific keys
Email provider	<code>EMAIL_PROVIDER</code> + provider-specific keys

8.2 Provider Configuration

SMS Providers (configured via `SMS_PROVIDER`):

Value	Provider	Notes
<code>mock</code>	None — logs to console	Development/testing only
<code>orange</code>	Orange Cameroun	+237 numbers only

Value	Provider	Notes
twilio	Twilio	International coverage
vonage	Vonage (Nexmo)	International coverage

Email Providers (configured via `EMAIL_PROVIDER`):

Value	Provider
mock	Logs to console — testing only
smtp	Gmail, Outlook, or custom SMTP server
resend	Resend.com API

9. Security & Compliance

9.1 Authentication & Authorisation

- All API endpoints require a valid **JWT (RS256)** access token
- Tokens are short-lived (15 minutes for agents, 24 hours for web sessions)
- Agent login uses a **time-based OTP** sent via SMS — valid for 5 minutes, single use
- RBAC is enforced at the middleware level on every request
- Agent sessions are bound to a specific device and shift window

9.2 Encryption

- All traffic is encrypted in transit via **TLS 1.2/1.3** (Let's Encrypt certificate)
- Database passwords and JWT keys are stored as encrypted GitHub Secrets
- OTP codes are stored as **HMAC-SHA256 hashes** with a secret pepper — never in plain text

9.3 Network Security

- Only ports 22, 80, and 443 are open on the server firewall
- The database is not exposed externally — accessible only within the Docker network
- The backend API is not directly reachable — all requests go through Nginx

9.4 Dependency Security

- Rust dependencies are audited on every CI run using `cargo-audit`
- Known vulnerabilities with no available fix are explicitly ignored with documented justification

9.5 Audit Logging

All administrative actions (user creation, session termination, submission approval/rejection) are recorded in the audit log with timestamp, actor, and action details.

10. Monitoring, Logging & Observability

IVISS uses a Prometheus + Grafana stack to collect and visualise metrics from both the frontend and backend. This section describes the tools, what is monitored, and how to access the dashboards.

10.1 Monitoring Stack

The following tools form the observability stack. They run as Docker containers alongside the application.

Tool	Purpose	Port
Prometheus	Metrics collection and storage	9090
Grafana	Dashboards and visualisation	3001
Metrics Server	Node.js bridge between frontend and Prometheus	9091

10.2 Backend Logging

The backend uses structured logging via the `tracing` crate. Log level is configurable via the `LOG_LEVEL` secret (`info` in production, `debug` for troubleshooting).

```
# View live backend logs
cd /opt/iviss
docker compose logs -f backend
```

10.3 Accessing Dashboards

Once the stack is running, the dashboards are accessible at the following URLs.

Dashboard	URL
Grafana	<code>https://<domain>:3001</code>
Prometheus	<code>https://<domain>:9090</code>

11. Backup & Disaster Recovery

11.1 Database Backup

 Automated database backups are not yet configured. This should be addressed before full production use.

Manual backup:

```
docker compose exec db pg_dump -U iviss_user iviss_prod >
backup_$(date +%Y%m%d).sql
```

Recommended approach (to be implemented): Daily `pg_dump` exported to S3 with 30-day retention.

11.2 Infrastructure Recovery

Because all infrastructure is defined as code, the entire server environment can be reproduced from scratch:

```
./infra/scripts/deploy.sh <domain> <email>
```

Recovery Time Objective (RTO): ~10 minutes for infrastructure

Recovery Point Objective (RPO): Dependent on backup frequency (manual at present)

12. Rollback & Incident Response

12.1 Rolling Back a Deployment

```
cd /opt/iviss
# Edit docker-compose.yml to use the previous version tag (e.g.
:v0.1.0)
docker compose pull
docker compose up -d
```

12.2 Detecting a Bad Release

- Health endpoint returns non-200: `curl https://<domain>/api/v1/health`
- Containers in `Restarting` state: `docker compose ps`
- Error spike in Grafana dashboard
- Agent login failures reported by field teams

12.3 Incident Severity Levels

Level	Description	Response time
P1 — Critical	System completely down	Immediate
P2 — High	Core feature broken (login, vehicle search)	< 1 hour
P3 — Medium	Non-critical feature degraded	< 4 hours
P4 — Low	Minor UI issue, cosmetic bug	Next release

13. Maintenance & Operational Procedures

13.1 Routine Operations

Task	Command
View all container statuses	<code>docker compose ps</code>
View live logs	<code>docker compose logs -f</code>
Restart a specific service	<code>docker compose restart backend</code>
Pull latest images and restart	<code>docker compose pull && docker compose up -d</code>
Connect to the database	<code>docker compose exec db psql -U iviss_user -d iviss_prod</code>

13.2 SSL Certificate Renewal

SSL certificates renew automatically every 90 days. To manually trigger renewal:

```
certbot renew
```

13.3 Shift Configuration

Agent login hours are controlled by `SHIFT_START_HOUR` and `SHIFT_END_HOUR` secrets (UTC+1, Africa/Douala). Changes require a redeployment.

14. Troubleshooting

Agents cannot log in / OTP not received

1. Check `SMS_PROVIDER` is set correctly
2. Check backend logs: `docker compose logs backend | grep sms`
3. If using Orange Cameroun, verify the phone number starts with `+237`
4. Verify the request is within the configured shift hours

401 Unauthorized errors on the frontend

1. Verify `JWT_PRIVATE_KEY_PEM` and `JWT_PUBLIC_KEY_PEM` are correctly formatted (single line, `\n` separators)
2. Redeploy to ensure the backend picked up the correct keys

Deployment fails — Terraform state checksum mismatch

1. Go to AWS DynamoDB → `iviss-terraform-lock` table (eu-central-1)
2. Delete the item with the matching `LockID`
3. Re-run the deployment

Docker images fail to pull

1. Verify `REGISTRY_TOKEN` has not expired
2. Generate a new GitHub PAT with `read:packages` permission and update the secret

Database connection errors

```
docker compose ps      # Verify db container is running
docker compose logs db  # Check for startup errors
```

Frontend shows blank page after deployment

```
docker compose restart frontend
```

15. Appendices

15.1 Glossary

Term	Definition
CI/CD	Continuous Integration / Continuous Deployment — automated pipeline for testing and deploying code

Term	Definition
Docker	Containerisation platform — packages the application and its dependencies into isolated units
Terraform	Infrastructure as Code tool — defines and provisions cloud resources from configuration files
Ansible	Configuration management tool — automates server setup and application deployment
JWT	JSON Web Token — a signed token used to authenticate API requests
OTP	One-Time Password — a temporary 6-digit code sent via SMS for agent login
RBAC	Role-Based Access Control — restricts system access based on user roles
SemVer	Semantic Versioning — a versioning standard using MAJOR.MINOR.PATCH format
GHCR	GitHub Container Registry — private Docker image storage
PWA	Progressive Web App — a web application installable on mobile devices
RTO	Recovery Time Objective — target time to restore service after an incident
RPO	Recovery Point Objective — maximum acceptable data loss in case of failure

15.2 Useful Links

Resource	URL
GitHub Repository	https://github.com/skyengpro/iviss
GitHub Releases	https://github.com/skyengpro/iviss/releases
GitHub Actions	https://github.com/skyengpro/iviss/actions
Container Registry	https://github.com/orgs/skyengpro/packages
AWS Lightsail Console	https://lightsail.aws.amazon.com

15.3 Key File Locations on Server

Path	Contents
/opt/iviss/	Application root — docker-compose.yml and .env
/opt/iviss/.env	Runtime environment configuration
/var/log/nginx/	Nginx access and error logs

15.4 Emergency SSH Access

```
ssh -i iviss-key.pem ubuntu@<server-ip>
```

The private key is output by `terraform output private_key` and saved at `infra/ansible/iviss-key.pem` during deployment.

Document Version

Version: 1.0

Last Updated: April 30, 2026

Author: IVISS Development Team

For the latest version of this guide, check the Help section in the IVISS back-office or contact your system administrator.

Welcome to IVISS. We're here to make your work safer, faster, and more effective.